

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE INGENIERÍA DE SOFTWARE

Informe Técnico Individual
Análisis de Consistencia y Replicación en
Bases de Datos Distribuidas

Código: TA-IND-02

Estudiante: Pedro Leonardo Castro López

Equipo PFC: BCEL — SGA Escuela de Educación Básica “Provincias Unidas”

Asignatura: Aplicaciones Distribuidas (ISR-701)

Unidad: 3 — Bases de Datos Distribuidas

Nivel / Paralelo: Séptimo Nivel — Paralelo “A”

Docente: Ing. Gleiston Cicerón Guerrero Ulloa

Período académico: 2026 – 2027

Fecha: Julio de 2026

Resumen

El presente informe analiza críticamente el modelo de consistencia y la estrategia de replicación implementados en el Sistema de Gestión Académica (SGA) desarrollado por el equipo BCEL para una escuela de educación básica en zona rural. El sistema adopta una arquitectura de microservicios comunicados mediante gRPC; sin embargo, su capa de persistencia se apoya en una única instancia de PostgreSQL segmentada en cuatro schemas, sin ningún mecanismo de replicación diseñado a nivel de aplicación. Se caracteriza dicho modelo como consistencia fuerte de tipo ACID, clasificable como PC/EC bajo la formulación PACELC. Se evalúan dos alternativas —replicación primario-respaldo y base de datos por microservicio con consistencia eventual— contrastadas mediante seis criterios técnicos. Se concluye que la elección de consistencia fuerte es adecuada, dada la criticidad de calificaciones y matrículas y la baja concurrencia del sistema; no obstante, la ausencia de replicación constituye una limitación crítica, pues la base única representa un punto único de fallo que contradice la disponibilidad diferenciada que justifica la propia arquitectura de microservicios. Se recomienda adoptar replicación líder-seguidor como mitigación, preservando la consistencia fuerte en las escrituras.

1. Introducción

El Sistema de Gestión Académica (SGA) desarrollado por el equipo BCEL para la Escuela de Educación Básica “Provincias Unidas” es una aplicación distribuida que automatiza los procesos de matriculación, registro de calificaciones, control de asistencia y generación de reportes de una institución rural ubicada en El Empalme, provincia del Guayas. El sistema atiende a aproximadamente 344 estudiantes, 14 docentes y personal administrativo, y está organizado en cuatro microservicios (Principal, Docente, Secretaría y Soporte Técnico) que se comunican internamente mediante gRPC.

Un aspecto determinante del dominio es su contexto operativo: la institución se ubica en una zona rural con conectividad de fibra óptica sujeta a cortes, lo que convierte la disponibilidad y la tolerancia a fallos en requisitos sensibles. Al mismo tiempo, la información gestionada —calificaciones y matrículas— es de alta criticidad, donde un dato incorrecto resulta inaceptable. Esta tensión entre *disponibilidad* y *consistencia* constituye el eje del presente análisis.

El objetivo de este informe es caracterizar el modelo de consistencia y la estrategia de replicación realmente implementados en el SGA, ubicarlos dentro del espectro de garantías de las bases de datos distribuidas y evaluarlos de forma crítica frente a alternativas, respondiendo a la pregunta orientadora: **¿es el modelo de consistencia elegido el óptimo para el dominio del sistema?**

2. Marco Teórico

2.1. Modelos de consistencia

La consistencia de réplicas no es una propiedad binaria, sino un espectro de garantías ordenadas por su fuerza semántica [3]. En el extremo más fuerte, la **linealizabilidad** (o consistencia fuerte) garantiza que toda lectura devuelve la escritura más reciente, como si existiera un único orden global coherente con el tiempo real. La **consistencia secuencial** preserva un orden común entre procesos, pero sin la garantía de tiempo real. La

consistencia causal respeta el orden de las operaciones relacionadas por causa–efecto, permitiendo que las concurrentes se observen en distinto orden. La **lectura de los propios cambios** asegura que una sesión siempre observe sus propias escrituras. Finalmente, la **consistencia eventual** solo garantiza que, en ausencia de nuevas actualizaciones, todas las réplicas converjan con el tiempo. Un modelo más fuerte exige mayor coordinación entre nodos, lo que incrementa la latencia y reduce la disponibilidad; un modelo más débil admite mayor disponibilidad y menor latencia, a costa de lecturas potencialmente desactualizadas [3].

2.2. Teoremas CAP y PACELC

El teorema CAP establece que, ante una partición de red, un sistema distribuido no puede garantizar simultáneamente consistencia y disponibilidad [2]. Dado que las particiones son inevitables en un entorno distribuido, la decisión real se reduce a elegir entre consistencia (CP) o disponibilidad (AP) durante la partición. La formulación PACELC supera esta lectura al añadir que, incluso en ausencia de particiones (*Else*), persiste un compromiso entre latencia y consistencia [1]: garantizar consistencia fuerte obliga a coordinar réplicas, lo que añade latencia. Así, un sistema puede clasificarse como PA/EL o PC/EC según sus decisiones bajo partición y en operación normal.

2.3. Estrategias de replicación

La replicación consiste en mantener copias de los datos en varios nodos. Las estrategias principales son la replicación primario–respaldo (líder–seguidor), la multi–líder y la replicación sin líder por quórum [3]. De forma transversal, la propagación puede ser síncrona —que ofrece mayor consistencia a costa de latencia y disponibilidad— o asíncrona —que privilegia disponibilidad y latencia, resultando típicamente en consistencia eventual. Los sistemas de referencia industrial ilustran los extremos del espectro: Dynamo prioriza la disponibilidad mediante replicación sin líder y consistencia eventual [4], mientras que Spanner ofrece consistencia externa apoyándose en sincronización temporal mediante relojes atómicos [5].

2.4. Estado del arte

La gestión de datos en arquitecturas de microservicios continúa siendo un problema abierto. Laigner *et al.* [6], en un estudio sobre el estado de la práctica, evidencian que numerosos sistemas industriales conservan bases de datos compartidas pese a haber adoptado microservicios, y señalan la gestión de la consistencia entre servicios como uno de los desafíos menos resueltos del área. De la lectura conjunta de las fuentes se desprende una tensión relevante: los sistemas que definen los extremos del espectro —Dynamo y Spanner— fueron concebidos para operar a escala global, donde la coordinación entre réplicas constituye un costo dominante. Sin embargo, en sistemas de baja concurrencia dicho costo se diluye, de modo que el argumento habitual para relajar la consistencia pierde fuerza. La cuestión pertinente, por tanto, no es si la consistencia fuerte resulta demasiado costosa, sino si la infraestructura que la sostiene ofrece la disponibilidad que el dominio exige.

3. Análisis del modelo de consistencia del SGA

El SGA implementa una arquitectura de microservicios —Principal (Java/Spring Boot), Docente (Django), Secretaría (Java/Spring Boot) y Soporte Técnico— que se comunican internamente mediante gRPC. No obstante, la distribución ocurre en la **capa de servicios**, no en la **capa de datos**: todos los microservicios comparten una **única instancia de PostgreSQL**, separada lógicamente en cuatro schemas (`sga_principal`, `sga_docente`, `sga_secretaria` y `sga_soporte`). La topología resultante se ilustra en la Figura 1.

Este diseño corresponde al patrón *shared database* (base compartida con separación por schemas), identificado en la literatura reciente como un antipatrón frecuente en migraciones hacia microservicios [6]. Al residir toda la información en un solo motor relacional, las garantías transaccionales ACID de PostgreSQL aseguran **consistencia fuerte**: cualquier lectura devuelve siempre el último valor escrito, sin posibilidad de divergencia entre copias, dado que no existen réplicas de los datos [3].

Conviene precisar que la comunicación mediante gRPC no altera esta caracterización. Aunque los microservicios intercambian información por invocaciones remotas —por ejemplo, cuando el servicio Principal solicita calificaciones al servicio Docente—, dichas invocaciones no replican datos: son consultas que terminan resolviéndose sobre la misma base subyacente. El acoplamiento de datos, por tanto, permanece intacto.

En cuanto a la estrategia de replicación, el sistema **no aplica ninguna** a nivel de aplicación: opera sobre una instancia única, por lo que no existe esquema líder–seguidor, multi-líder ni quórum. Aunque el motor se ejecute sobre un proveedor gestionado en la nube que podría ofrecer redundancia a nivel de infraestructura, dicha redundancia es transparente a la aplicación, no fue diseñada por el equipo y no constituye una estrategia de replicación en sentido arquitectónico: el sistema sigue percibiendo un único punto de acceso. La base de datos permanece, por tanto, como un **punto único de fallo**.

Bajo el teorema CAP, al no existir múltiples nodos de datos gestionados por la aplicación, no hay particiones internas que resolver; el sistema privilegia por diseño la **Consistencia** [2]. En términos de PACELC se clasifica como **PC/EC**: prioriza la consistencia tanto ante un eventual fallo como en operación normal, aun a costa de la latencia y la disponibilidad [1]. Esta decisión se alinea con la criticidad de los datos académicos, donde un valor incorrecto resulta inaceptable.

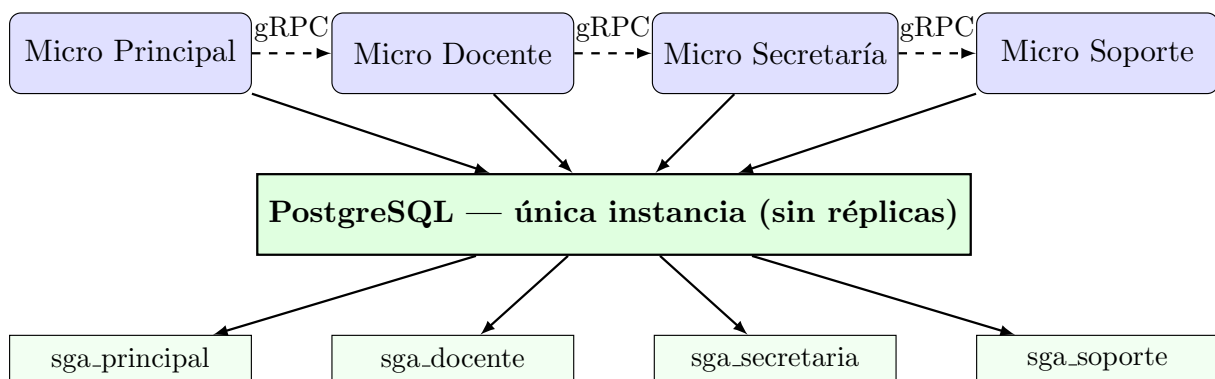


Figura 1: Topología del SGA: cuatro microservicios distribuidos sobre una única instancia de PostgreSQL separada por schemas, sin replicación gestionada por la aplicación.

4. Evaluación de alternativas

Para valorar si el modelo actual es el más adecuado, se contrastan dos alternativas viables frente al diseño vigente.

4.1. Alternativa A: replicación primario–respaldo (líder–seguidor)

Consiste en mantener la instancia actual como nodo líder y añadir una o más réplicas seguidoras que reciban los cambios mediante *streaming replication*, mecanismo nativo de PostgreSQL [3]. Las escrituras continúan centralizadas en el líder —preservando la consistencia fuerte—, mientras que las lecturas pueden distribuirse hacia las réplicas. Si la propagación es síncrona se conserva consistencia fuerte a costa de latencia; si es asíncrona, las réplicas ofrecen consistencia eventual pero mejoran la disponibilidad de lectura y habilitan conmutación por error ante la caída del líder. Esta alternativa elimina el punto único de fallo sin alterar el modelo de datos ni la lógica de negocio.

4.2. Alternativa B: base de datos por microservicio con consistencia eventual

Cada microservicio administraría su propia base de datos independiente, eliminando el acoplamiento del patrón *shared database* [6]. La coherencia entre servicios dejaría de ser transaccional y se lograría mediante propagación asíncrona de eventos o transacciones compensatorias (*saga*), lo que implica adoptar **consistencia eventual** entre dominios [4]. Esta opción maximiza la autonomía y la disponibilidad de cada servicio, pero introduce complejidad significativa: gestión de conflictos, idempotencia y ventanas de inconsistencia visibles para el usuario.

4.3. Comparación

La Tabla 1 contrasta el modelo actual con ambas alternativas según seis criterios técnicos.

5. Discusión crítica

La pregunta orientadora admite una respuesta matizada: el modelo es **adecuado en su elección de consistencia, pero no óptimo en su estrategia de replicación**.

En cuanto a la consistencia, la decisión es correcta. El dominio académico gestiona calificaciones y matrículas, datos cuya exactitud es innegociable: una nota desactualizada o una matrícula duplicada tienen consecuencias administrativas reales. Además, la carga es reducida —alrededor de veinte usuarios concurrentes y 344 estudiantes—, por lo que no existe presión de escalabilidad que justifique relajar las garantías. Sacrificar consistencia para ganar latencia (la rama *EL* de PACELC) carecería de beneficio práctico y añadiría riesgo funcional [1].

La limitación se encuentra en la capa de replicación, y es más profunda de lo que aparenta. La justificación arquitectónica del proyecto sostiene que los microservicios permiten disponibilidad diferenciada: si el módulo docente falla, el sistema administrativo debe seguir operando. Sin embargo, al compartir todos los servicios una **única instancia de**

Tabla 1: Comparación del modelo actual del SGA frente a dos alternativas de consistencia y replicación.

Criterio	Actual: base compartida	A: Líder–seguidor	B: BD por servicio
Consistencia	Fuerte (ACID)	Fuerte en el líder; eventual en réplicas asíncronas	Eventual entre servicios
Disponibilidad	Baja: punto único de fallo	Media–alta: permite conmutación por error	Alta: cada servicio opera de forma autónoma
Latencia	Media: todo al nodo único	Baja en lecturas (réplicas)	Baja: acceso local por servicio
Tolerancia a particiones	Nula	Parcial: la réplica sigue sirviendo lecturas	Alta: los servicios operan aislados
Complejidad operativa	Baja	Media: réplicas y <i>failover</i>	Alta: eventos, sagas y conflictos
Clasificación PA-CELC	PC/EC	PC/EC (síncrona) o PA/EL (asíncrona)	PA/EL

base de datos, esa promesa no se cumple: la caída de PostgreSQL inutiliza simultáneamente a los cuatro microservicios. El sistema es distribuido en su capa de servicios, pero **monolítico en su capa de datos**, lo que revela una contradicción entre el objetivo declarado y el diseño implementado. Esta observación coincide con lo reportado en la literatura, donde la base compartida se identifica como un antipatrón que neutraliza el aislamiento de fallos perseguido por la descomposición [6].

El riesgo se agrava por el contexto operativo. El propio requisito no funcional del proyecto reconoce que la institución opera con fibra óptica sujeta a cortes; en consecuencia, la indisponibilidad no es un escenario hipotético sino recurrente. Un docente que no puede registrar asistencia durante un corte, o una secretaria que no puede consultar una matrícula, experimentan una falla total del sistema, pues no existe ningún nodo alternativo que sirva la información.

Como mitigación se propone adoptar la **Alternativa A**. Esta opción conserva la consistencia fuerte en las escrituras —preservando la garantía que el dominio exige— y elimina el punto único de fallo mediante réplicas de lectura y conmutación por error. Es una mejora de bajo costo conceptual, pues no altera el modelo de datos ni exige rediseñar la lógica de negocio.

Por el contrario, la **Alternativa B** se descarta como prematura. Aunque representa el patrón canónico de microservicios y maximiza la autonomía, su complejidad —sagas, idempotencia y resolución de conflictos— no se justifica para la escala de esta institución [6]; y admitir ventanas de inconsistencia sobre calificaciones sería funcionalmente inaceptable en el dominio educativo.

En síntesis, el sistema debe permanecer en la rama *PC/EC* para sus operaciones críticas, pero puede beneficiarse de relajar la consistencia únicamente en lecturas no críticas

—como la consulta de horarios— mediante réplicas asíncronas, adoptando un enfoque híbrido que mejora la disponibilidad sin comprometer la integridad académica.

6. Conclusiones

El SGA implementa un modelo de **consistencia fuerte** sostenido por las propiedades ACID de una única instancia de PostgreSQL segmentada en cuatro schemas, sin ninguna estrategia de replicación gestionada por la aplicación. Bajo PACELC, el sistema se ubica de forma inequívoca en la rama PC/EC.

Esta elección resulta **acertada respecto de la consistencia**: la criticidad de los datos académicos y la baja concurrencia del entorno hacen que las garantías transaccionales fuertes sean preferibles a cualquier relajación orientada a latencia o escalabilidad.

Sin embargo, la elección **no es óptima en conjunto**. La ausencia de replicación introduce un punto único de fallo que anula la disponibilidad diferenciada prometida por la arquitectura de microservicios y resulta especialmente riesgosa en un entorno rural con conectividad inestable.

La mitigación recomendada es incorporar **replicación primario-respaldo**, que preserve la consistencia fuerte en escrituras mientras habilita tolerancia a fallos y mejora la latencia de lectura. La migración a bases de datos por microservicio con consistencia eventual, si bien canónica, se considera prematura para la escala y las exigencias de integridad del dominio.

Declaración de uso de IA generativa

En la elaboración de este informe se utilizó la herramienta de inteligencia artificial generativa **Claude (Anthropic)** como apoyo puntual. Su uso se limitó a: (i) aclarar y explicar conceptos teóricos sobre modelos de consistencia, CAP/PACELC y estrategias de replicación; (ii) orientar la estructura del documento conforme a la rúbrica; y (iii) asistir en cuestiones de formato del código LaTeX, como la sintaxis del diagrama de topología y de la tabla comparativa. La identificación y caracterización del modelo realmente implementado en el SGA, la búsqueda y verificación de las referencias con sus respectivos DOI, la selección y el contraste de las alternativas, así como la postura crítica final, son producto del trabajo y el criterio del autor, basados en el conocimiento directo del sistema desarrollado por el equipo BCEL. El autor revisó, reescribió y validó la totalidad del contenido, asumiendo la responsabilidad integral del texto.

Referencias

- [1] D. J. Abadi, “Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story,” *Computer*, vol. 45, no. 2, pp. 37–42, 2012, doi: 10.1109/MC.2012.33.
- [2] S. Gilbert and N. Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,” *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002, doi: 10.1145/564585.564601.

- [3] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 3rd ed. distributed-systems.net, 2017.
- [4] G. DeCandia *et al.*, “Dynamo: Amazon’s Highly Available Key-value Store,” in *Proc. 21st ACM SIGOPS Symp. Operating Systems Principles (SOSP ’07)*, 2007, pp. 205–220, doi: 10.1145/1294261.1294281.
- [5] J. C. Corbett *et al.*, “Spanner: Google’s Globally Distributed Database,” *ACM Transactions on Computer Systems*, vol. 31, no. 3, pp. 1–22, 2013, doi: 10.1145/2491245.
- [6] R. Laigner, Y. Zhou, M. A. V. Salles, Y. Liu, and M. Kalinowski, “Data Management in Microservices: State of the Practice, Challenges, and Research Directions,” *Proceedings of the VLDB Endowment*, vol. 14, no. 13, pp. 3348–3361, 2021, doi: 10.14778/3484224.3484232.